Online Scheduling of CPU-NPU Co-inference for Edge AI Tasks

Xiancheng Lin[1], Rongkai Liu[1], Jiajie Xie[2], Qian Wei[1], Zhi Zhou[2], Xu Chen[2], Zhilan Huang[1], Gang Lu[1]
[1]Research Institute of China Telecom Co., Ltd., Guangzhou 510630, China
[2]Sun Yat-sen University, Guangzhou 510006, China

2023 IEEE Wireless Communications and Networking Conference (WCNC) | 978-1-6654-9122-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/WCNC55385.2023.10118755

***Abstract*—Edge AI is an emerging paradigm that leverages edge computing to pave the last mile delivery of artificial intelligence. To satisfy the stringent timeliness and energy-efficiency requirements of emerging edge AI tasks, specialized AI accelerator of Neural Processing Units (NPU) have been widely equipped by edge nodes. Compared to the traditional centralized processing units (CPU), NPU has better performance and energy-efficiency. However, these benefits come at the cost of reduced inference accuracy. As a result, existing coarse-grained scheduling mechanisms that schedule a whole DNN task to either the CPU or NPU are unable to make the best use of NPU. To address this issue, we propose an online NPU-CPU co-inference scheduling mechanism to schedule the DNN task at the fine-grained layer level, and thus to fully utilize the performance, accuracy, and power diversities of the NPU and CPU. By applying Lyapunov optimization to schedule the network layers dynamically, our proposed online scheduling mechanism is able to ensure the real-time inference speed and cap the long-term time-averaged power consumption, while still approximately minimizes the long-term inference accuracy loss. Via rigorous theoretical analysis as well as realistic trace-driven simulations, we demonstrate the effectiveness of our proposed online scheduling mechanism.**

I. Introduction

Driven by the breakthroughs in deep learning, the recent years have witnessed an unprecedented booming of AI applications ranging from smart city, smart retail, video surveillance to Industrial-Internet-of-Things (IIoT). As a critical driver that boosts these AI applications, big data is increasingly originated at the ubiquitous IoT devices such as video cameras. Notably, shipping bulk data from the devices to the remote cloud would greatly deteriorate the application performance and cost-efficiency. In response, edge intelligence [1] has emerged as a new paradigm to push AI tasks to the networks edges to process data in closer proximity to users and IoT devices.

Since DNN-based AI tasks are typically resource-demanding, how to run them smoothly under resource constraints has been the central problem for edge intelligence. To answer this problem in the positive, both software- and hardware-based solutions have been proposed. At the software side, model compression [2] methods can be applied to shape the resource demand of DNN models. Specifically, by applying model compression methods, such as model weight pruning, quantization, and compact network design [3], the compressed model is with less number of weights, smaller model size, and thus less resource demanding. While at the hardware side, specialized AI accelerators such as NPU have been emerged to argument the capability of traditional general purposed processor of CPU [4]. Compared to the traditional CPU, NPU has better performance (in terms of inference speed) and energy-efficiency when running data- and computational-intensive DNN inference tasks. Moreover, compared to the software-based solutions such as model compression, NPU has better applicability since we do not need to refine the DNN model when the application changes.

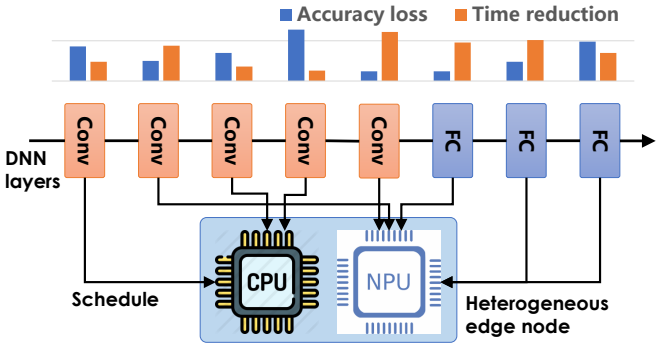


Fig. 1. An illustration of schedule of CPU-NPU co-inference for edge DNN inference tasks. Here the histogram plots the accuracy loss and processing time reduction for running each layer of the DNN on NPU.

While recognizing the superiority of NPU on the inference performance and power-efficiency, we should also note that these benefits come at the cost of deteriorated inference accuracy. Specifically, NPU has limitations on the precision of floating-point numbers, which are usually represented by low-precision 16 bits or 8 bits rather than 32 bits of CPU [4]. As a result, existing coarse-grained task scheduling mechanisms that schedule a whole DNN inference task to either the CPU or the NPU are unable to make the best use of the NPU and CPU. To fully utilize the diversities of performance, accuracy, and power-efficiency of the heterogeneous NPU-CPU architecture, in this paper we advocate a fine-grained scheduling mechanism which adaptively schedules each DNN layer rather than the whole DNN model to the CPU and NPU. Specifically, if we detect a DNN layer which incurs slight accuracy loss but significant running time reduction when migrating from the CPU to the NPU, then we should schedule it to the NPU for execution.

*This work was supported in part by the National Science Foundation of China under Grants 62172454, 61972432, and U20A20159, the Guangdong Basic and Applied Basic Research Foundation under Grants 2021A1515011912 and 2021B151520008. The corresponding author is Zhi Zhou.

978-1-6654-9122-8/23/$31.00 ©2023 IEEE
Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 06:52:27 UTC from IEEE Xplore. Restrictions apply.

However, when performing fine-grained CPU-GPU co-inference, minimizing the accuracy loss under real-time performance constraint and long-term power cap is rather challenging. Specifically, the long-term power cap is time-coupling, i.e., involving future system information in terms of the future execution time of the CPU. However, in realistic edge computing environments with shared-CPU, the execution time each layer on the CPU typically fluctuates over time and thus cannot be readily predicted. Then, without knowing the future CPU execution time as a priori knowledge, how can we guarantee the current decisions are able to maximize the time-averaged inference accuracy, while still maintaining the long-term power cap? Second, even with an offline setting where all the future information is given as a priori knowledge, the corresponding problem is proven to be NP-hard. This further complicates the design of an online algorithm that dynamically schedules the DNN layer based on the historical and current system information.

Fortunately, by exploiting the structural properties of the problem, we address the above challenges by blending the advantages of Lyapunov optimization for online algorithm design and an efficient approximate algorithm for the minimum knapsack problem. Specifically, by applying Lyapunov optimization theory, we temporally decompose the long-term time-coupling problem into a series of one-shot subproblems, each can be solved online, without requiring any future information. For each subproblem, we further show that since the Lyapunov optimization theory incorporates the long-term power cap into a unified objective, it can be transformed to the classical minimum knapsack problem and efficiently solved by a greedy algorithm. The upshot of our resulting online algorithm is that, it is easily tunable by a control parameter V that represents the relative importance of accuracy loss minimization versus power cap enforcement, and facilitates a provable $[O(1/V), O(V)]$ accuracy-power tradeoff. Via rigorous theoretical analysis and empirical trace-driven evaluations, we verify the efficacy of our proposed online algorithm.

II. Related Work

With the recent emergence of edge intelligence, exploiting heterogeneous edge resources to accelerate the inference DNN tasks has been a hot topic. In this regard, the first line of research efforts attempt to offload partial inference workload to one or more heterogeneous edge or cloud nodes via data or model partition. For example, Mao *et al.* proposed to partition the input figure into multiple parts and dispatch them to various edge nodes, such that the inference can be accelerated via data-parallelism [5]. Similarly, Hu *et al.* suggested to partition the model into several parts, and executes the submodels at different edge nodes in parallel. Along a different way, some recent works [6], [7] have also explored how to accelerate DNN inference via cloud-edge synergy, i.e., offload the computational-intensive part of the DNN model from the edge to the powerful cloud.

More recently, specialized hardware such as GPU and NPU has been increasingly integrated into edge nodes to argument the capability for processing AI tasks. As a result, how to efficiently coordinate the heterogeneous devices such as CPU, GPU, and/or NPU within the same edge node to accelerate DNN tasks is becoming a new problem. To answer this problem, Tan *et al.* [4] schedule the DNN layers between CPU and NPU to jointly optimize the accuracy and performance. Some recent works [8], [9] study how to map DNN layers to heterogeneous GPU and CPU, with the goal of optimizing the performance and energy consumption under throughput constraint. Finally, Lane *et al.* [2] compress and decompose the DNN model, and allocate the decomposed submodels to heterogeneous processors in an energy-efficient manner.

III. System Model and Problem Formulation

A. System Model

System Overview: As shown in Fig. 1, we consider a heterogeneous edge node that integrating both the general CPU and specialized NPU to serve emerging AI applications such as live video analytics for Internet-of-Vehicles (IoV). In such applications, the input data, i.e., video frames are continuously captured to perceive the physical environment and obtain effective driving information through DNN model inference. For the dynamic arrival of the video frames to the edge node, we characterize it with a frame rate of F FPS (frame-per-second), meaning that the time interval between two consecutive video frames is $\frac{1}{F}$ second. In this paper, we denote the set of dynamically arrived video frames by $\mathcal{T} = \{1, 2, \cdots, T\}$. For the DNN model that processes the video frames, as it has a layered structure as shown in Fig. 1, we denote $\mathcal{I} = \{1, 2, \cdots, I\}$ as the layer set.

CPU-NPU Co-Inference Model: as motivated in Sec. I, to fully leverage the performance and accuracy diversities of the heterogeneous CPU-NPU architecture, we should carefully schedule the execution of DNN model inference at the fine-grained layer level. To model this scheduling decision, we use a binary variable $x_i(t)$ to indicate whether the i-th layer of the DNN model is scheduled to the NPU ($x_i(t) = 1$) or not ($x_i(t) = 0$) when processing the video frame $t \in \mathcal{T}$. To characterize the inference accuracy loss incurred by the NPU, we use A_i to denote the accuracy loss of running each *single* layer $i \in \mathcal{I}$ on the NPU (in regard to the CPU inference accuracy). For the processing time of each layer, we use N_i to denote the processing time of layer $i \in \mathcal{I}$ on the NPU. Considering the sharing nature (e.g., by some other tasks and background OS tasks) of the CPU, we adopt a time-varying constant $C_i(t)$ to denote the processing time of layer $i \in \mathcal{I}$ on the CPU when processing the video frame $t \in \mathcal{T}$. Without loss of generality, we assume that $C_i(t) > N_i, \forall i \in \mathcal{I}, t \in \mathcal{T}$.

Performance and Power Cap Constraints: notably, when performing fine-grained CPU-NPU co-inference, we should ensure that each video frame is finished before the next frame arrives. This indicates that the total processing time of each video frame should not exceed the time interval between two consecutive video frames, i.e., $\sum_{i=1}^{I}[x_i(t)N_i + (1 - x_i(t))C_i(t)] \leq \frac{1}{F}$. For the power consumption, we use P^N and P^C to denote the dynamical power of the NPU and CPU

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 06:52:27 UTC from IEEE Xplore. Restrictions apply.

when performing DNN model inference. In practice, to maintain the hardware reliability and life span, computer systems are typically enforced with a long-term time-averaged power cap. In this paper, we denote the long-term time-averaged power cap by $\bar{P}$, then, the time-averaged power consumption $\frac{1}{T}\sum_{t=1}^{T}\sum_{i=1}^{I}\frac{x_i(t)N_iP^N+(1-x_i(t))C_i(t)P^C}{1/F}$ of the edge node should not exceed $\bar{P}$, i.e., $\frac{1}{T}\sum_{t=1}^{T}\sum_{i=1}^{I}[x_i(t)N_iP^N+(1-x_i(t))C_i(t)P^C]\leq\frac{\bar{P}}{F}$.

B. Problem Formulation

Ideally, when optimizing the schedule of CPU-NPU, we want to minimize the accuracy loss while stile maintaining the performance and power cap constraints. Unfortunately, exactly measuring or profiling the accuracy loss $f(x_1,x_2,\cdots,x_I)$ under different schedules $(x_1,x_2,\cdots,x_I)$ is unpractical, since we face an exponential solution space. To address this challenge, following the recent work [10], we minimize the sum of the accuracy loss of each layer, i.e., $\sum_{t=1}^{T}\sum_{i=1}^{I}A_ix_i(t)$. While this simplified objective does not necessarily equal to the actual accuracy loss under the schedule $(x_1,x_2,\cdots,x_I)$, minimizing it can effectively reduce the actual accuracy loss. Then, we formulate the problem of minimizing the long-term accuracy loss under performance and power cap constraints as follows:

$$\min \quad \frac{1}{T}\sum_{t=1}^{T}\sum_{i=1}^{I}A_ix_i(t) \tag{1}$$

$$\text{s.t.} \quad \sum_{i=1}^{I}[x_i(t)N_i+(1-x_i(t))C_i(t)]\leq\frac{1}{F}, \tag{2}$$

$$\frac{1}{T}\sum_{t=1}^{T}\sum_{i=1}^{I}[x_i(t)N_iP^N+(1-x_i(t))C_i(t)P^C]\leq\frac{\bar{P}}{F}, \tag{3}$$

$$x_i(t)\in\{0,1\}. \tag{4}$$

Challenge analysis: solving the above optimization problem faces two challenges. Firstly, this long-term problem is time-coupling and involves future uncertain information in terms of the future processing time $C_i(t)$. However, in realistic edge computing environments with shared-CPU, $C_i(t)$ typically fluctuates over time and thus cannot be readily predicted. Then, without knowing $C_i(t)$ as a priori, how can we simultaneously minimize the time-averaged inference accuracy loss, while still maintaining the long-term power cap? Secondly, even in the offline case in which $C_i(t)$ is given as a priori, the corresponding problem is still NP-hard, as it can be reduced from the minimum knapsack problem [11] which is known to be NP-hard.

IV. An Online Optimization Framework

In response to the challenge of the long-term optimization problem (1), we fuses the power of Lyapunov optimization and approximate algorithm design to develop and analyze an online optimization framework.

A. Problem Decomposition with Lyapunov Optimization

As discussed, a primary challenge of the optimization problem (1) — involving the future uncertainty — is posed by the time-coupling power cap constraint in Eq. (3). Interestingly, from the perspective of queuing theory, the constraint in Eq. (3) can be viewed as the queue stability control paradigm, i.e., the time-averaged arrival rate $\lim_{T\to\infty}\frac{1}{T}\sum_{t=1}^{T}\sum_{i=1}^{I}[x_i(t)N_iP^N+(1-x_i(t))C_i(t)P^C]$ of a queuing process cannot exceed the constant service rate $\frac{\bar{P}}{F}$. Fortunately, the queue stability problem has been well studied in literature [12], [13], and Lyapunov optimization is a powerful technique to maintain the stability in an online manner, without requiring any future knowledge or statistical distribution of the uncertain information.

Towards transforming the long-term power cap constraint in Eq. (3) into a well-studied queue stability control problem, we introduce a virtual queue $\boldsymbol{Q}$ whose queue backlog (i.e., queue length) $Q(t)$ at each time slot t evolves according to the following queuing dynamics:

$$Q(t+1)=\max\Big\{Q(t)+\sum_{i=1}^{I}\big[x_i(t)N_iP^N+(1-x_i(t))C_i(t)P^C\big]-\frac{\bar{P}}{F},0\Big\}. \tag{5}$$

Initially, we set $Q(0)=0$. For this virtual queue, $\sum_{i=1}^{I}[x_i(t)N_iP^N+(1-x_i(t))C_i(t)P^C]$ represents dynamical arrival process, while $\frac{\bar{P}}{F}$ denotes the constant service process. Then, maintaining the power cap constraint in Eq. (3) can be interpreted as enforcing that the time-averaged arrival rate $\lim_{T\to\infty}\frac{1}{T}\sum_{t=0}^{T-1}\sum_{i=1}^{I}[x_i(t)N_iP^N+(1-x_i(t))C_i(t)P^C]$ is no larger than the constant service rate $\frac{\bar{P}}{F}$. Moreover, it is intuitive that the queue backlog $Q(t)$ can be interpreted as the historical accumulation of the gap (i.e., power deficit) between the actual power consumption and total power cap during the time interval $[0,t-1]$. And a larger value of $Q(t)$ implies that the actual power consumption during the time interval $[0,t-1]$ exceeds the power cap $\bar{P}$. This observation suggests that we could maintain the long-term power cap constraint in Eq. (3) by enforcing $Q(t)$ stable (i.e., avoiding from excessive growth), as suggested by the following Lemma 1.

Lemma 1: *If the virtual queue $\boldsymbol{Q}$ keeps stable over time, i.e., $\lim_{T\to\infty}\frac{\mathbb{E}\{Q(T)\}}{T}$=0, then the long-term power cap constraint in Eq (3) is satisfied.*

The proof of Lemma 1 is given in our online technical report [14]. Lemma 1 indicates that by introducing the virtual queue $\boldsymbol{Q}$, we can transform the original long-term power cap constraint into a well-studied queue stability control problem. Towards applying Lyapunov optimization to finitely bound $Q(t)$ and thus to maintain the stability of the virtual queue, we adopt the quadratic Lyapunov function [12] which is defined as $L(Q(t))=\frac{1}{2}Q^2(t)$.

Intuitively, to keep the virtual queue stable, i.e., with a finite upper bound over time, we should persistently push the Lyapunov function towards a lower value state. To this end,

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 06:52:27 UTC from IEEE Xplore. Restrictions apply.

we further introduce $\Delta(Q(t))$ as the one-step conditional Lyapunov drift: $\Delta(Q(t)) = \mathbb{E}\{L(Q(t+1)) - L(Q(t))|Q(t)\}$. The drift $\Delta(Q(t))$ measures the change of the Lyapunov function between two consecutive slots. Clearly, by minimizing the drift per slot to restrain the Lyapunov function, we can prevent the queue backlog of the virtual queue from unbounded growth, and thus maintain the queue stability.

In the sense of Lyapunov control, our two-fold objectives of minimizing the time-averaged inference accuracy loss while still enforcing the long-term power cap, is now transformed to minimize the weighted sum of the Lyapunov drift and the inference loss (referred to as the drift-plus-loss expression) at each time slot t:

$$\Delta(Q(t)) + V\mathbb{E}\{\sum_{i=1}^{I} A_i x_i(t)|Q(t)\}. \tag{6}$$

The control parameter $V(\geq 0)$ represents a knob of the stability-loss tradeoff, i.e., how much we shall emphasize stabilizing the virtual queue compared to minimizing the accuracy loss at time slot t. More excitingly, the drift-plus-loss expression further indicates that, the original long-term optimization problem has now been transformed to a series of real-time subproblems which only involve the current time slot t. This enables us to *make online decisions on layer scheduling based on the current information, without requiring the future information as a priori.*

B. An Adaptive Online Algorithm

By applying Lyapunov optimization, we transform the original long-term problem (1) to a set of real-time drift-plus-loss minimization subproblems (6) that can be solved in a online manner. However, directly minimizing Eq. (6) requires to handle the implicit $\max[*]$ terms. In response, we seek to design an online algorithm to minimize its supremum bound derived in the following Lemma 2.

Lemma 2: *At each time slot t, for any queue backlog $Q(t)$ and any layer schedule, the drift-plus-loss $\Delta(Q(t)) + V\mathbb{E}\{\sum_{i=1}^{I} A_i x_i(t)|Q(t)\}$ satisfies the following inequality, where the finite constant $\Phi \triangleq \frac{1}{2}[(\frac{\bar{P}}{F})^2 + IP_{\max}^2]$, and $P_{\max} = \max_{i,t} C_i(t)P^C$.*

$$\Delta(Q(t)) + V\mathbb{E}\{\sum_{i=1}^{I} A_i x_i(t)|Q(t)\} \leq Q(t)[\sum_{i=1}^{I} C_i(t)P^C - \frac{\bar{P}}{F}]$$
$$+\Phi + \mathbb{E}\Big\{\sum_{i=1}^{I} x_i(t)[VA_i + Q(t)(N_iP^N - C_i(t)P^C)]|Q(t)\Big\}. \tag{7}$$

The proof of Lemma 2 is given in our online technical report [14]. Lemma 2 derives a supremum bound of drift-plus-loss expression at each time slot t, which only involves the current information and layer schedule decision. By omitting the constant terms Φ, $Q(t)\frac{\bar{P}}{F}$ and $Q(t)C_i(t)P^C$ in the right-hand-side of Eq. (7), we obtain the following real-time and deterministic optimization problem at each time slot t.

$$\min \quad \sum_{i=1}^{I} x_i(t)[VA_i + Q(t)(N_iP^N - C_i(t)P^C)] \tag{8}$$
$$\text{s.t.} \quad \sum_{i=1}^{I} x_i(t)[C_i(t) - N_i] \geq \sum_{i=1}^{I} C_i(t) - \frac{1}{F},$$
$$x_i(t) \in \{0, 1\}.$$

Interestingly, we can observe that the above transformed and simplified problem (8) is a standard $0-1$ min-knapsack problem. While this problem is proven to be NP-hard, it can be efficiently solved by a greedy heuristic proposed in [11]. For ease of presentation, we let $H_i(t) = VA_i + Q(t)(N_iP^N - C_i(t)P^C)$, $W_i(t) = C_i(t) - N_i$ and $\bar{C}(t) = \sum_{i=1}^{I} C_i(t) - \frac{1}{F}$. The basic idea of the greedy heuristic is to first sort the DNN layers in nondecreasing order of their cost-to-performance ratios. Here the cost-to-performance ratio of an DNN layer $i \in \mathcal{I}$ is defined as $\frac{H_i(t)}{W_i(t)}$. So after sorting, we have $\frac{H_1(t))}{W_1(t)} \leq \frac{H_2(t))}{W_2(t)} \leq \cdots \leq \frac{H_I(t))}{W_I(t)}$. Then, the greedy heuristic traverses through the sorted DNN layers to find a DNN layer k_1 which satisfies $\sum_{i=1}^{k_1} W_i(t) < \bar{C}(t) \leq \sum_{i=1}^{k_1+1} W_i(t)$. By finding this layer k_1, we construct a candidate solution, and the sublist $\mathcal{I}_1 = (1, 2, \cdots, k_1)$ represents the NPU-friendly layers to be scheduled to the NPU in the candidate solution.

Step 1: now starting from $k_1 + 2$ we traverse through the remaining layers and let k_2 be the next layer satisfying $\sum_{i=1}^{k_1} W_i(t) + W_{k_2}(t) < \bar{C}(t)$. So for all the DNN layers $j \in \{k_1+2, \cdots, k_2-1\}$, the inequality $\sum_{i=1}^{k_1} W_i(t) + W_j(t) \geq \bar{C}(t)$ holds. We denote all the DNN layers from $k_1 + 1$ to $k_2 - 1$ as the CPU-friendly layers and let the first set of CPU-friendly layers be $\mathcal{N}_1 = (k_1 + 1, \cdots, k_2 - 1)$. Then, all sets $\mathcal{I}_1 \cup \{j\}, \forall j \in \{k_1 + 1, \cdots, k_2 - 1\}$, are candidate layers set to be scheduled to the NPU.

Step 2: we continue to find the first DNN layer k_3 satisfying $\sum_{i=1}^{k_1} W_i(t) + \sum_{i=k_2}^{k_3} W_i(t) < \bar{C}(t) \leq \sum_{i=1}^{k_1} W_i(t) + \sum_{i=k_2}^{k_3+1} W_i(t)$. By defining a second set of NPU-friendly layers $\mathcal{I}_2 = (k_2, \cdots, k_3)$, the set $\mathcal{I}_1 \cup \mathcal{I}_2 \cup \{k_3 + 1\}$ also consists a candidate solution.

We repeat the step 1 and step 2 until the end of the list using k_{2n+1} instead of k_1 and k_{2n+2} instead of k_2 in the n-th iteration. Then, the final solution generated by the greedy heuristic will be the candidate with smallest objective value. It is easy to see that the computational complexity of sorting step and the later steps is $O(I \log I)$ and $O(I)$, respectively.

Finally, for the whole procedure of the proposed the online scheduling of CPU-NPU co-inference, it is shown in Algorithm 1.

C. Performance Analysis

We are now ready to analyze the optimality of presented online optimization framework. To begin with, we first analyze the performance of the greedy heuristic for the decoupled NP-hard problem (8) in Lemma 3.

Lemma 3: *The greedy heuristic in line 3-15 in Algorihtm 1 has an approximation ratio of* 2 *for problem (8).*

The proof of Lemma 3 is referred to [11]. Based on Lemma 3, we further derive the performance guarantee of the whole online Alogrithm 1, in terms of a well-balanced tradeoff between the total accuracy loss and the virtual queue stability.

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 06:52:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Online Scheduling of CPU-NPU Co-Inference

Initialize the control parameter V and queue backlog $Q(0) = 0$;
for each time slot $t \in \{0, 1, 2, \cdots, T\}$ **do**
  For each DNN layer $i \in \mathcal{I}$, calculate $H_i(t)$, $W_i(t)$, $\bar{C}(t)$, and denote the candidate solution set for problem (8) as CSL;
  Sort DNN layers in nondecreasing order of $\frac{H_i(t)}{W_i(t)}$;
  Find the first index k_1 which satisfies $\sum_{i=1}^{k_1} W_i(t) < \bar{C}(t) \leq \sum_{i=1}^{k_1+1} W_i(t)$;
  Let $\mathcal{I}_1 = (1, 2, \cdots, k_1)$ be the first NPU-friendly layer set and add $\mathcal{I}_1 \cup \{k_1 + 1\}$ into CSL;
  repeat
    /* ▷ Step 1 */
    Find the first index $k_2 > k_1 + 1$ which satisfies $\sum_{i=1}^{k_1} W_i(t) + W_{k_2}(t) < \bar{C}(t)$ and $\sum_{i=1}^{k_1} W_i(t) + W_j(t) \geq \bar{C}(t), \forall j \in \{k_1 + 2, \cdots, k_2 - 1\}$;
    Let $\mathcal{N}_1 = (k_1 + 1, \cdots, k_2 - 1)$ be the first CPU-friendly layer set and add $\mathcal{I}_1 \cup \{j\}, \forall j \in \{k_1 + 2, \cdots, k_2 - 1\}$ into CSL;
    /* ▷ Step 2 */
    Find the first index $k_3 \geq k_2$ which satisfies $\sum_{i=1}^{k_1} W_i(t) + \sum_{i=k_2}^{k_3} W_i(t) < \bar{C}(t) \leq \sum_{i=1}^{k_1} W_i(t) + \sum_{i=k_2}^{k_3+1} W_i(t)$;
    Let $\mathcal{I}_2 = (k_2, \cdots, k_3)$ be the second NPU-friendly layer set and add $\mathcal{I}_1 \cup \mathcal{I}_2 \cup \{k_3 + 1\}$ into CSL;
  until The end of the layer list using k_{2n+1} instead of k_1 and k_{2n+2} instead of k_2 in the n-th iteration;
  Calculate the objective value of each candidate solution in CSL and schedule the layer set with the smallest objective value on NPU in time slot t;
  Update the power-aware virtual queue $Q(t+1)$ according to Eq. (5).
end for

Theorem 1: *For any $V > 0$, Algorithm guarantees that the gap between the achieved time-averaged total accuracy loss and the offline optimum Π^{opt} for problem (1) is within Φ/V, where Φ is a constant defined in Lemma 2.*

$$\lim_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\{\sum_{t=1}^{T} \sum_{i=1}^{I} A_i x_i(t)\} \leq 2\Pi^{opt} + \frac{\Phi}{V}. \tag{9}$$

Moreover, Algorithm 1 also ensures that the virtual queue is strongly stable over time slots:

$$\lim_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\{Q(t)\} \leq \frac{\Phi + 2V\Pi^{opt}}{\epsilon}, \tag{10}$$

where ϵ is a constant which represents the gap between the time-averaged power consumption by some stationary strategy and the power cap $\frac{\bar{P}}{F}$.

Insight: Theorem 1 provides a strong deterministic guarantee of the upper bounds on the time-averaged accuracy loss and queue backlog, both of which can be flexibly tuned by the control parameter V. Specifically, Eq. (9) indicates that the gap between the long-term time-averaged accuracy loss achieved by our online optimization algorithm and the offline optimum Π^{opt} is within $O(1/V)$. However, according to Eq. (10), the increase of the control parameter V can also deteriorate the upper bounds of queue backlog, which in return deteriorates the enforcement of the long-term power cap. The proof of Theorem 1 is given in our online technical report [14].

V. Performance Evaluation

A. Experimental Setup

Application and Platform: we envisage a live edge video analytics application building on top of the object detection model of YOLO small. The frame rate F of the live video stream is set to be 10 frame-per-second (FPS). We consider a scheduling period of one hour, i.e., with a number of 36000 time slots. At each time slot, a frame of the video is captured and to be processed by the edge node. For the edge node, following the recent literature on NPU scheduling [4], we consider the smartphone of HUAWEI mate 10 pro, which is equipped with 6 GB memory, octa-core CPU (4×2.4 GHz and 4×1.8 GHz) and a Cambricon NPU.

Inference Performance, Accuracy Loss, and Power: in this paper, we set up the inference latency N_i as well as the accuracy loss A_i of running each YOLO small model layer on the Cambricon NPU based on the recent empirical measurements conducted by [4]. To simulate the time variability of the CPU inference latency $C_i(t)$, we add a random variable that follows a normal distribution to the inference latency in the case of full occupation (also conducted by [4]). For the NPU and CPU power consumption, we set $P^N = 1W$ and $P^C = 5W$, respectively, based on the recent empirical measurement studies [15]. Finally, the long-term time-averaged power cap $\bar{P}$, is set to be $2.5W$.

Baselines: here we compare our proposed online algorithm with two baselines: (1) the pure NPU approach which processes all the video frameworks and all the layers of the YOLO small model on the NPU, to satisfy the real-time performance goal and long-term time-averaged power cap constraint. (2) The coarse-grained CPU-NPU co-scheduling mechanism CoarCoInf which schedules the whole DNN model to either the CPU or the NPU for each video frame to minimize the long-term time-averaged accuracy loss under the power cap constraint. For our find-grained CPU-NPU co-inference scheduling mechanism, we refer it as FineCoInf.

B. Evaluation Results

Inference Accuracy Loss: Fig. 2 plots the time-averaged accuracy loss for various values of V under different schemes. We make the following observations: (1) as V increases, accuracy loss achieved by our proposed scheme FineCoInf declines dramatically and approaches a minimum level. This is consistent with Theorem 1 in that our proposed scheme can approach the minimal accuracy loss with a diminishing gap $O(1/V)$ (captured by Eq. (9)). The accuracy loss diminishes as

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 06:52:27 UTC from IEEE Xplore. Restrictions apply.

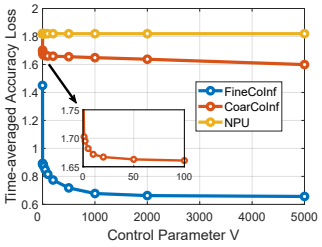


Fig. 2. The effect of control parameter V on the time-averaged accuracy loss

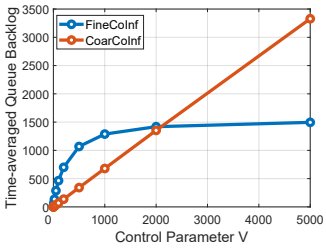


Fig. 3. The effect of control parameter V on the time-averaged queue backlog

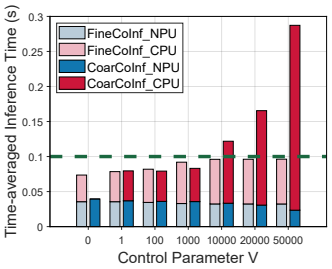


Fig. 4. The effect of control parameter V on the time-averaged inference time

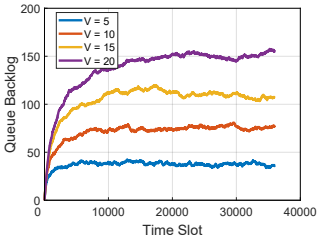


Fig. 5. Fluctuation of the real-time queue backlog over time

V increases, however, as the accuracy loss will eventually converges to the minimum. (2) Our proposed FineCoInf scheme performs much better than the CoarCoInf approach and the NPU approach. This rationale is that our FineCoInf scheme fully makes use of the temporal and spatial diversities of the inference performance, accuracy, and power consumption with fine-grained scheduling at the model layer level.

Queue Stability: Fig. 3 plots the time-averaged queue backlog for different values of V under our FineCoInf scheme and the baseline CoarCoInf. The figure shows that as V increases, the backlog increases in a sub-linear manner, which is consistent with the worst-case bound captured by Eq. (10) in Theorem 1. Along with Fig. 2, this reflects the tradeoff between the queue stability and the accuracy loss. Further, when comparing between our FineCoInf approach and CoarCoInf, we can see that the fine-grained scheduling at the model layer level not only reduce the accuracy loss (as shown in Fig. 2), but also improves the queue stability by reducing the queue backlog.

Inference Performance: Fig. 4 plots the time-averaged inference time as well as the breakdown of the inference time for different values of V under our FineCoInf scheme and the baseline CoarCoInf. We observe that our FineCoInf scheme can always ensures that the average inference time does not exceed the performance goal of $0.1s$ (i.e., $\frac{1}{F}$ in the right-hand-side of Eq. (2)). While as the control parameter V increases, the average inference time increases dramatically such the layers would aggressively scheduled to the CPU to minimize the accuracy loss.

Adaptive Control: we finally examine the capability of online adaptive control that has been interpreted in Sec. IV-B. Specifically, Fig. 5 depicts the fluctuation of the queue backlog $Q(t)$ under various values of V. From the figure we observe that the queue backlog fluctuates frequently. The intuition is that a large queue backlog greatly increases the risk of violating the queue stability and power cap, making our framework strive to reduce the queue congestion and prevent the violation of the power cap. Then as the queue backlog reduces, our framework will switch to minimize the accuracy loss, leading to the increased queue congestion again. It can also be seen from Fig. 5 that the real-time queue backlog has a larger peak value with a larger V, this quantitatively corroborates Eq. (10) that bounds the queue backlog in Theorem 1.

VI. Conclusion

In this paper, we present an online optimization framework of CPU-NPU co-inference scheduling for edge AI tasks. To fully exploit the performance, power and accuracy diversities of the heterogeneous CPU-NPU architecture, the presented optimization framework carefully schedule the execution of DNN model inference at the find-grained layer level. By fusing the power of Lyapunov optimization and approximate algorithm design to schedule the DNN layers, the presented optimization framework is able to simultaneously ensure the real-time inference speed and cap the long-term time-averaged power consumption, while still approximately maximizes the long-term time-averaged inference accuracy. The efficacy of the presented optimization framework is verified via both rigorous theoretical analysis and extensive trace-driven simulations.

References

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, 2019.

[2] N. D. Lane, S. Bhattacharya, P. Georgiev, C. Forlivesi, L. Jiao, L. Qendro, and F. Kawsar, "Deepx: A software accelerator for low-power deep learning inference on mobile devices," in *Proc. of ACM IPSN*, 2016.

[3] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.

[4] T. Tan and G. Cao, "Efficient execution of deep neural networks on mobile devices with npu," in *Proc. of ACM IPSN*, 2021.

[5] J. Mao, Z. Yang, W. Wen, C. Wu, L. Song, K. W. Nixon, X. Chen, H. Li, and Y. Chen, "Mednn: A distributed mobile system with enhanced partition and deployment for large-scale dnns," in *Proc. of IEEE/ACM ICCAD*, 2017.

[6] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge ai: On-demand accelerating deep neural network inference via edge computing," *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447–457, 2019.

[7] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, "Spinn: synergistic progressive inference of neural networks over device and cloud," in *proc. of ACM Mobicom*, 2020.

[8] D. Kang, J. Oh, J. Choi, Y. Yi, and S. Ha, "Scheduling of deep learning applications onto heterogeneous processors in an embedded device," *IEEE Access*, vol. 8, pp. 43 980–43 991, 2020.

[9] W. Kang, K. Lee, J. Lee, I. Shin, and H. S. Chwa, "Lalarand: Flexible layer-by-layer cpu/gpu scheduling for real-time dnn tasks," in *Proc. of IEEE RTSS*, 2021.

[10] R. Han, Q. Zhang, C. H. Liu, G. Wang, J. Tang, and L. Y. Chen, "Legodnn: block-grained scaling of deep neural networks for mobile vision," in *Proc. of ACM Mobicom*, 2021.

[11] J. Csirik, "Heuristics for the 0-1 min-knapsack problem," *Acta Cybernetica*, vol. 10, no. 1-2, pp. 15–20, 1991.

[12] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool, 2010.

[13] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. of IEEE INFOCOM*, 2018.

[14] Online Technical Report. [Online]. Available: https://1drv.ms/b/s!Ar9mS_s-frkZhe5izkPZak_FcKr11w?e=cfGe0E

[15] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

Authorized licensed use limited to: Southwest University of Science and Technology. Downloaded on October 08,2026 at 06:52:27 UTC from IEEE Xplore. Restrictions apply.